

Indexing

Ze gaan hoogstwaarschijnlijk niet om theorie vragen zoals "wat is een index seek/scan" etc.

Ze gaan waarschijnlijk zoals in voorbeeldexamen vragen

execution plan

Table scan --> AVOID THIS. you search through all the pages of the tables

Index scan --> index is used but it is scanned from the start till the searched records are found.

Index Seek --> tree structure of index is used, resulting in very fast data retrieval.

create an index

```
create nonclustered index EmpLastName_Incl_TitleONemployee(lastname, firstname)
INCLUDE (title);
```

making indexes best practises

avoid functions

```
-- BAD
SELECT EmployeeID, Birthdate
FROM Employee
WHERE Year (BirthDate) = 1980;

-- GOOD
SELECT EmployeeID, Birthdate
FROM Employee
WHERE BirthDate >='1980-01-01' AND BirthDate < '1981-01-01';
```

```
-- BAD
SELECT productname, unitsinstock*price
FROM Products
ORDER BY unitsinstock*price

-- GOOD
-- By creating an index on a computed column or by creating a materialized view
(see further) you can simulate the effect of a function based index.
ALTERTABLE dbo.Product ADD RetailValue AS(Unitsinstock* Price * 1.5);

create index retailvalueindex on product(retailvalue) include (productname);

select productname, retailvalue
```

```
from Product
order by RetailValue;
```

avoid calculations, isolate columns

```
--BAD
SELECT EmployeeID, FirstName, LastName
FROM Employee
WHERE Salary * 1.10 > 100000;

--GOOD
SELECT EmployeeID, FirstName, LastName
FROM Employee
WHERE Salary > 100000/1.10;
```

avoid ANY and ALL

```
-- BAD
SELECT lastname, firstname, birthdate
from Employee
where BirthDate >= all(select BirthDate from Employee)

--GOOD
SELECT lastname, firstname, birthdate
from Employee
where BirthDate = (select max(BirthDate) from Employee)
```

Index for equality first, then for ranges

```
select lastname, birthdate country
from Employee
where BirthDate >= '1980-01-01' and BirthDate <= '1990-12-31' and Country
='Canada'

create index IdxCountryBirthdate on Employee(country, birthdate)
```

others

- Check the SQL code that is generated by your ORM tool or framework
- Avoid dynamic SQL whenever possible
- Use bind variables
- Execute joins in the database
- Avoid unnecessary joins

Materialized view

